NATIONAL UNIVERSITY OF OSTROH ACADEMY
Department of Information Technologies and Data Analytics

Lab 4-5.
AI agent for a web quality audit: accessibility, technical SEO, and Core Web Vitals

Modern Web Development: HTML, CSS, JS/TS

Deadline: 30.09.2026

1. Aim

Explore the main approaches to a comprehensive quality audit of a web page: accessibility, technical SEO, and performance. Master Chrome DevTools, Lighthouse, and automated accessibility checks for finding problems. Learn to use an AI agent as a tool for analyzing, classifying, and fixing problems in a web page, while keeping control over whether the proposed solutions are correct.

2. Objectives and learning outcomes

After completing this lab, you must be able to:

- Understand the core principles of web accessibility and the structure of WCAG 2.2, in particular which criteria appeared in version 2.2.
- Check a web page with the keyboard, through Chrome DevTools, and with the Accessibility Tree.
- Understand the purpose of ARIA and the cases where using it is justified.
- Apply automated accessibility checks and explain their limitations.
- Carry out a basic technical SEO audit of a web page.
- Understand the purpose of JSON-LD / Schema.org, Open Graph, `robots.txt`, and `sitemap.xml`, and why the location of `robots.txt` matters.
- Understand the difference between crawling and indexing.
- Understand the purpose and main metrics of Core Web Vitals (LCP, INP, CLS) and the difference between lab data and field data.
- Use Lighthouse and Chrome DevTools to analyze performance and set correct measurement conditions.

- Use an AI agent for a comprehensive audit of a web page.
- Critically verify AI recommendations against the documentation and the results of automated and manual checks.
- Carry out the **audit** → **verification** → **fix** → **re-audit** cycle.

3. Lab tasks

1. Run a series of experiments on accessibility, automated testing, technical SEO, and Core Web Vitals, relying on the official documentation [1]–[15] and experiments in the browser.
2. Run an initial audit of the provided starter project using DevTools and automated tools.
3. Run an AI-assisted comprehensive audit and produce a list of the problems and recommendations it finds.
4. Verify the AI recommendations against the documentation and the results of your own experiments.
5. Fix the justified problems and run the audit again.
6. Write a research report that shows not only the final result, but also the process of **finding** → **verifying** → **fixing** → **re-checking**.
7. Defend your work orally.

> **Important!** The practical part is done on the provided starter project, which contains deliberately introduced problems of various kinds. Your task is to find them on your own, **confirm** them, and fix them — not to reproduce a ready-made solution.

4. Research tasks

Before starting the practical part, study the theory and run the experiments. Add the results (observations and conclusions) to your report. Verify every decision against sources [1]–[15].

4.1. Web accessibility and WCAG 2.2

Study:

1. Web Content Accessibility Guidelines (WCAG) 2.2 — W3C
2. WCAG 2 at a Glance — W3C
3. What's New in WCAG 2.2 — W3C WAI

Research the four WCAG principles (Perceivable, Operable, Understandable, Robust), the concept of a Success Criterion, the conformance levels A / AA / AAA, and also keyboard accessibility, focus, accessible names, contrast, forms and labels, and semantic HTML.

Separately, identify **what appeared specifically in WCAG 2.2**: 2.4.11 Focus Not Obscured (Minimum), 2.5.7 Dragging Movements, 2.5.8 Target Size (Minimum), 3.2.6 Consistent Help, 3.3.7 Redundant Entry, 3.3.8 Accessible Authentication (Minimum) — and the fact that criterion 4.1.1 Parsing was declared obsolete and removed in version 2.2.

For contrast, record the specific threshold values, because at the defense you will need a number, not an assessment like "it looks a bit pale":

Criterion	What it measures	Minimum for AA
1.4.3 Contrast (Minimum)	regular text	4.5 : 1
1.4.3 Contrast (Minimum)	large text (≥ 24px or ≥ 18.66px bold)	3 : 1
1.4.11 Non-text Contrast	field borders, icons, UI states	3 : 1
2.5.8 Target Size	pointer target size	24 × 24 CSS px

Experiment

Pick one interactive element on the page and check it with the mouse, the keyboard, and DevTools → Accessibility Tree. Compare the results:

- whether the element can be reached with the keyboard;
- whether it has an accessible name;
- which role the browser sees;
- whether the accessibility representation matches the element's actual purpose.

4.2. Accessibility in Chrome DevTools

Study:

4. Accessibility features reference — Chrome DevTools

Research: the Accessibility Tree, ARIA attributes, Computed Accessibility Properties, the Source Order Viewer, and the contrast check.

The Accessibility Tree is a subset of the DOM Tree and contains the elements relevant to presenting the content to assistive technologies.

Experiment

For **two** elements of different types (for example, a heading and an interactive element), open the **Accessibility** tab and determine the Role, the Name, the available properties, and the position in the tree.

Separately, check the order in which the `Tab` key moves through the page and conclude whether the focus order matches the logic of the page. If the order differs from the visual one, use the **Source Order Viewer** to see why.

4.3. Automated accessibility audit

Study:

5. Lighthouse — Chrome for Developers
6. axe-core — Accessibility engine

Research:

- which accessibility problems can be detected automatically;
- what the `violations`, `passes`, and `incomplete` results mean;
- why an automated audit cannot fully determine the accessibility of a web page;
- which problems must be checked manually.

Two facts that matter for the conclusions of this lab. First, the Accessibility category in Lighthouse is built on axe-core, so Lighthouse and the axe DevTools extension will give the same result for a large share of the rules — they are not two independent checks. Second, the set of automated rules mainly covers WCAG 2.0 / 2.1 criteria at levels A and AA; most of the new 2.2 criteria from §4.1 are not checked automatically at all.

> Checking with a screen reader (NVDA, VoiceOver) is outside the scope of this lab. Manual checking here means the keyboard and the Accessibility Tree.

Experiment

Run an accessibility audit on the starter project and record: the problems found, the elements they concern, the type / level of the problem, and the tool's recommendation.

For at least two problems, do a manual check in DevTools and determine whether the problem is really present. Separately, find **at least one problem on the page that the automated tool did not show**.

Conclusion: which problems can be reliably automated, and which need a developer's involvement?

4.4. AI-assisted accessibility audit

You can give the AI agent the HTML, CSS, JavaScript, the Lighthouse results, the axe results, and screenshots of individual parts of the page.

Example prompt:

```
Analyze this web page in terms of WCAG 2.2. Do not rewrite the code right away.
First produce a list of potential accessibility problems. For each, give:
1) the element; 2) the problem; 3) the relevant WCAG principle or Success
Criterion; 4) how to verify it; 5) the recommended fix.
Separately mark the problems that can be checked automatically and the problems
that need a manual check.
```

Research:

1. Does the AI find problems that Lighthouse or axe did not detect?
2. Does the AI identify the relevant WCAG criteria correctly?
3. Does the AI suggest unnecessary ARIA attributes?
4. Can the AI's recommendations contradict native HTML semantics?
5. Which recommendations must be verified against the documentation before you make changes?

> **The main rule:** an AI recommendation is not proof that a problem exists.

Typical examples where this rule is tested: the advice "add `aria-label` to all buttons" — for a button with ordinary visible text, this attribute is unnecessary and only overwrites the accessible name; the advice "add `role="button"` to this `<div>`" — the question is why not use a native `<button>` instead, which provides the role, focus, and keyboard handling without any attribute.

4.5. Technical SEO audit and structured data

Study:

7. Introduction to structured data markup in Google Search — Google Search Central
8. Schema Markup Validator — schema.org
9. Rich Results Test — Google

Research: structured data, Schema.org, JSON-LD, `@context`, `@type`, and the properties of structured data. Google recommends JSON-LD specifically, if that format fits the capabilities of the project.

To check the markup, use both tools and pay attention to the difference between them: the Schema Markup Validator [8] checks the syntax and conformance to the Schema.org vocabulary for any type, while the Rich Results Test [9] shows only the types for which Google generates rich results. Valid JSON-LD that does not produce rich results is not an error.

Experiment

Analyze the starter project and determine: whether it makes sense to use structured data; which Schema.org type matches the content of the page; which properties can be described; and which data can be taken directly from the content of the page.

Separately, check **whether the markup already on the page matches what is actually written in the content**.

Conclusion: why must structured data describe the actual content of the page?

4.6. Open Graph, robots.txt, and sitemap.xml

Study:

10. The Open Graph protocol
11. Introduction to robots.txt — Google Search Central
12. Build and Submit a Sitemap — Google Search Central

Open Graph. Explore the purpose of the main properties:

```
<meta property="og:title" content="...">
<meta property="og:description" content="...">
<meta property="og:type" content="website">
<meta property="og:url" content="...">
<meta property="og:image" content="...">
```

Open Graph represents a web page as an object in the social graph. Explain what `og:title` defines, what role `og:description` plays, what `og:image` is for, and why `og:url` must match the page's actual URL.

robots.txt. Explore the `User-agent`, `Allow`, `Disallow`, and `Sitemap` directives, and also the difference between crawling and indexing. Google defines `robots.txt` as a mechanism for controlling which URLs a crawler may request; it is **not** a mechanism for removing a page from Google Search — `noindex` exists for that.

sitemap.xml. Explore `<urlset>`, `<url>`, `<loc>`, `<lastmod>`, the URL requirements (absolute, same origin), and the relationship between a sitemap and `robots.txt`.

Experiment

Create a `robots.txt` and a `sitemap.xml` for the starter project, then check: whether the files are in the right place; whether they contain correct data; whether `robots.txt` blocks needed resources or pages; and whether the sitemap contains only relevant URLs.

> **Where exactly to check.** `robots.txt` is honored by search engines only at the root of an origin — `https://example.com/robots.txt`. For a project page, GitHub Pages serves the site at `https://<username>.github.io/<repository>/`, so the file `.../<repository>/robots.txt` is **not** the `robots.txt` of that origin: it remains an ordinary static file, and the crawler does not read it. Vercel gives you your own origin, at whose root `robots.txt` works as it should — so do the §4.6 and §5.1 checks on the Vercel deployment. This is the same reason `robots.txt` does not remove a page from the index: both questions are about what exactly the crawler reads and **where** it reads it.

4.7. Core Web Vitals and performance audit

Study:

13. Web Vitals — web.dev

14. Core Web Vitals workflows with Google tools — web.dev
15. Interaction to Next Paint (INP) — web.dev

The current set of Core Web Vitals includes three metrics. The recommended thresholds are evaluated at the **75th percentile of field data**:

Metric	What it measures	Recommended threshold
LCP	loading of the main content	≤ 2.5 s
INP	responsiveness to interaction	≤ 200 ms
CLS	visual stability	≤ 0.1

> **INP is not measured by a standard Lighthouse run.** It is a field metric: it is computed from real user interactions, so an ordinary (navigation) audit that only loads the page does not return it, and a new learning site will not have CrUX data either. The Lighthouse **Timespan** mode will show INP for the interactions you perform during the recording — but that is a single interaction of yours, not the 75th percentile of field data. **TBT** is a lab proxy only for thread blocking during loading: it will not see a slow click handler. Measure the actual interaction delay in DevTools → **Performance**: record a trace, perform the interaction, and find the long task in the event handler.

Measurement conditions

For the "before / after" comparison in §5.5 to make sense, both runs must be done the same way:

1. the same URL — the same deployment, not `file://` and not a dev server;
2. an incognito window, with browser extensions disabled;
3. the same throttling preset and the same device mode;
4. **three runs, with the median in the report**.

The spread between runs on unchanged code often exceeds the effect of a real fix. If the conditions are not fixed, the §5.5 table shows nothing — and this is the first thing that will be checked at the defense.

Experiment

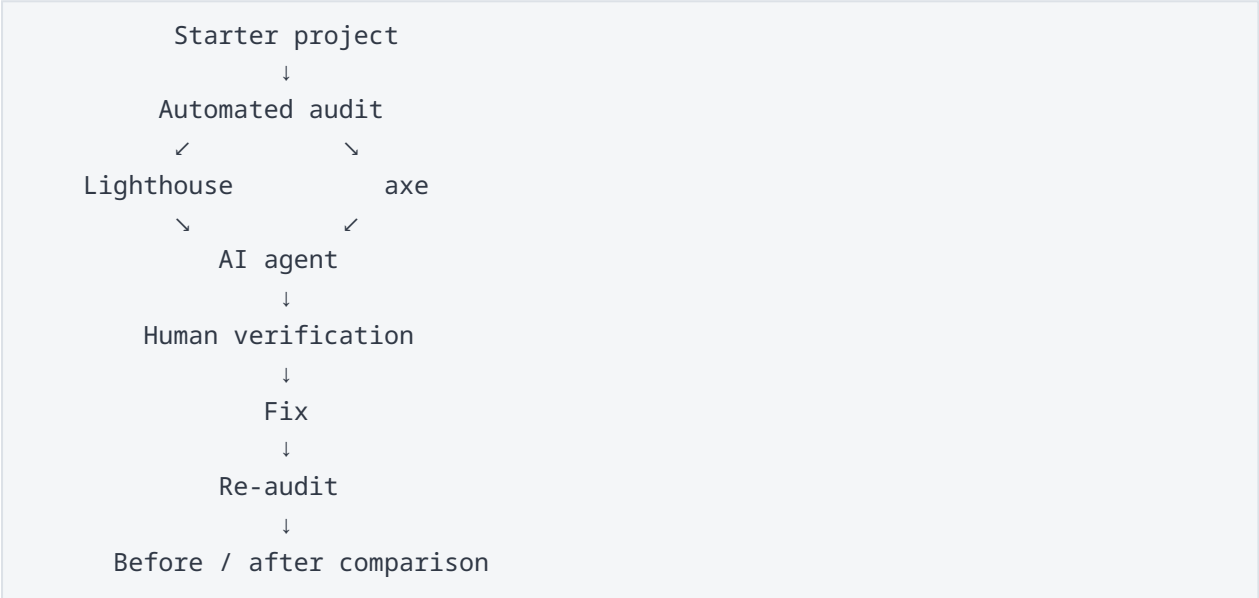
Run a Lighthouse Performance audit and explore LCP, CLS, TBT, FCP, Speed Index, and also the Insights and Diagnostics sections.

Find at least one **cause** of a performance problem and test it experimentally: determine which element is the LCP element and what exactly shifts the layout. Compare the results before and after the change.

5. Practical part: AI-assisted web quality audit

Using the provided starter project, carry out a comprehensive audit of accessibility, technical SEO, and performance.

The starter project contains deliberately introduced problems of various kinds. Your task is not just to find them with the help of AI or automated tools, but to **confirm** each problem, determine its cause, justify the fix, and verify the result after making the changes.



5.1. Initial audit

Before making any changes, run the initial check and record the results in your report.

Accessibility. Semantic HTML and landmarks, heading hierarchy, the document's `lang`, the page `<title>` (2.4.2), accessible names, links and buttons, keyboard navigation, visible focus (2.4.7), images, forms and labels, form error messages (3.3.1), contrast, ARIA, the Accessibility Tree.

Technical SEO. This check splits into two parts:

- *regression* — what was already the topic of Lab 3: `<title>`, meta description, heading structure, link text, `canonical`;
- *the new layer* — Open Graph, JSON-LD, `robots.txt`, `sitemap.xml`, and also `<meta name="robots">` on the page.

Performance. LCP, CLS, TBT, the rest of the Lighthouse diagnostics and — most importantly — the causes of problems, not just the numbers. Follow the measurement conditions from §4.7.

Canonical origin

The work is published in two places, so the page will be available at two different URLs, and `canonical` and `og:url` cannot point to both at the same time. Choose **one** canonical origin, use it in `canonical` and `og:url` on all pages, and be able to explain your choice. Do the `robots.txt` and `sitemap.xml` checks on the Vercel deployment (see §4.6).

5.2. AI-assisted audit

Give the AI agent the HTML, CSS, JavaScript, the Lighthouse results, the accessibility scanner results and, if needed, screenshots.

The AI must not rewrite the project right away. First it must produce an **audit report** that gives the following for each problem found:

Field	Contents
Category	Accessibility / SEO / Performance
Problem	Description of the problem
Evidence	What the conclusion is based on
Severity	Low / medium / high
Recommendation	How to fix it
Verification	How to verify the result

In your report, record **which AI agent and which version** you used, and the date of the run: without this the result cannot be reproduced.

5.3. Verifying AI recommendations

Select at least **5 AI recommendations**. For each, determine:

- whether the problem is confirmed;
- which tool can verify it;
- which source can justify the solution;
- whether the recommendation will be implemented;
- if not — why it was rejected.

Pay particular attention to recommendations about ARIA, semantic HTML, accessible names, WCAG, structured data, `robots.txt`, and performance.

5.4. Fixing the problems

Implement the justified fixes. The minimum required is:

1. at least **2 accessibility problems**;
2. at least **2 technical SEO problems**;
3. at least **1 performance problem** — or a justified demonstration that the recorded problem does not need a change;
4. at least **1 AI recommendation that you rejected**, with a justification and a link to a source.

Item 4 is a required result, not a wish: a submission without a single rejected recommendation does not demonstrate the main skill of this lab.

The main structure and visual concept of the starter project must be preserved.

5.5. Re-audit

After making the changes, run the Lighthouse audit, the accessibility scan, keyboard testing, the Accessibility Tree check, the technical SEO check, and the performance audit again — under the same conditions as the initial one (§4.7).

Check	Before	After
Accessibility (manual)	…	…
Lighthouse Accessibility	…	…
Automated (axe)	…	…
LCP	…	…
CLS	…	…
TBT	…	…
Performance diagnostics	…	…
Technical SEO	…	…

Example of a filled-in table. The numbers are made up and given only to show the format; your results will depend on the deployment and the device. State the measurement conditions above the table, and in the cells give not only the number but also a short reason for the change.

> Conditions: Vercel deployment, Chrome 140, incognito, Lighthouse → Mobile, "Slow 4G" preset, median of three runs.

Check	Before	After
Accessibility (manual)	11 problems, 5 of them not found automatically	2 problems remain (justification in the report)

Check	Before	After
Lighthouse Accessibility	71	100 — but this does not mean there are no problems (see the row above)
Automated (axe)	6 violations (14 elements), 1 "needs review"	0 violations, 1 "needs review" checked manually
LCP	6.4 s — LCP element identified, cause recorded	2.2 s
CLS	0.27 — two sources of shift	0.02
TBT	0 ms — TBT does not see a slow click handler	0 ms; interaction delay measured in Performance: 310 → 40 ms
Performance diagnostics	7 items	2 items — deliberately not fixed, with justification
Technical SEO	Lighthouse SEO 58; 6 problems by manual check	Lighthouse SEO 100; JSON-LD, robots.txt, sitemap.xml verified

Conclude which problems were eliminated, which remain, which turned out to be false positives, and which need a manual check.

> **Important!** You must demonstrate not only positive results. Being able to explain why an AI recommendation was rejected, or why an automated tool cannot definitively determine a problem, is part of the lab.

6. Report requirements

Email subject:

```
[Web] Lab 4-5: AI Agent for a Web Quality Audit: Accessibility, Technical SEO, and Core Web Vitals
```

In the email, include links to:

```
GitHub repository: https://github.com/<username>/<repository>
GitHub Pages: https://<username>.github.io/<repository>/
Vercel: https://<project>.vercel.app/
```

The report must contain:

1. the results of the research tasks;
2. links to the sources you used;
3. the results of the initial audit, the **measurement conditions** (§4.7), and the chosen canonical origin with a justification (§5.1);
4. the AI prompt, the name and version of the AI agent used, and the date of the run;
5. the results of the AI audit (the audit report using the §5.2 table);

6. an analysis of at least 5 AI recommendations, at least one of them rejected — with a justification and a link to a source;
7. a description of the fixes you made, and a justification for the problems you deliberately did not fix;
8. screenshots of DevTools and of the automated checks;
9. a before / after comparison of the results (the §5.5 table);
10. a conclusion about the audit results (§5.5).

> **Important!** The report does not replace the result: you hand in **both** the report **and** the deployed, fixed starter project. A report without a working deployment is not considered.

7. Oral defense

Handing in this lab includes an oral interview about your practical implementation. Even if the AI agent proposed the audit and part of the fixes, you must be able to explain every change, show how it was verified, and answer questions with screen sharing.

Sample questions:

Accessibility

- Open the Accessibility Tree for a specific element. Which role and accessible name does it have, and where do they come from?
- Navigate the page using only the keyboard. Can you reach all interactive elements? Is the focus visible at every step?
- Which problem did axe / Lighthouse find? Show it in the HTML and name the WCAG criterion it relates to.
- Which problem did you find **manually** that the automated tool did not? Why does the result of an automated audit not mean that the page is fully accessible?

AI audit

- Which AI fix did you accept, and how did you verify that it is correct? Which recommendation did you reject, and why?

Technical SEO

- Which `canonical` did you choose, and why exactly this URL?
- Show the JSON-LD. Why did you choose this Schema.org type, and does it match the content of the page?
- Show `robots.txt` and `sitemap.xml`. On which origin do they actually work? How does `robots.txt` differ from `noindex`?

Core Web Vitals

- Which element is the LCP element? What LCP value did you get before and after optimization, and under what conditions did you measure?
- What is the cause of CLS on the page, if it occurs?
- Why is INP not in an ordinary Lighthouse report? Show in DevTools → Performance how you found the cause of the interaction delay.

8. Grading

1. A successful defense — **4 points**.
2. An additional **2 points** if either of the following applies:
    - you submit by **September 30, 2026**, OR
    - you do the work in class.